

day16笔记

一、自定义聊天机器人

1、common.py文件

```
import json

def get_llm_response(client, *, system_prompt='', few_shot_prompt='', user_prompt='',
                    model='qwen3-max', stream=False):
    # 定义聊天列表
    messages = []
    if system_prompt:
        messages.append({'role': 'system', 'content': system_prompt})
    if few_shot_prompt:
        messages += json.loads(few_shot_prompt)
    if user_prompt:
        messages.append({'role': 'user', 'content': user_prompt})

    # 补齐资源
    resp = client.chat.completions.create(
        model=model,
        messages=messages,
        stream=stream
    )

    # 判断是否开启了流模型
    if not stream:
        return resp.choices[0].message.content
    return resp
```

2、示例代码

```
from openai import OpenAI
from common import get_llm_response
import streamlit as st

def get_answer(question: str):
    client = OpenAI(base_url=base_url, api_key=api_key)
    stream = get_llm_response(client, model=model_name, user_prompt=question, stream=True)
    for chunk in stream:
        yield chunk.choices[0].delta.content or ''

# 侧边栏区域
with st.sidebar:
    api_vendor = st.radio(label='请选择模型服务器商', options=['ChatTongyi', 'DeepSeek'])
    if api_vendor == 'ChatTongyi':
        base_url = 'https://dashscope.aliyuncs.com/compatible-mode/v1'
        model_options = ['qwen-plus', 'qwen-max', 'qwen-turbo', 'qwen3-max']
    elif api_vendor == 'DeepSeek':
        base_url = 'https://api.deepseek.com'
```

```

model_options = ['deepseek-chat', 'deepseek-reasoner'] # deepseek-reasoner R1模型

# 模型名称
model_name = st.selectbox(label='请选择你要使用的模型', options=model_options)
# 输入key, 推荐使用密码框, 因为可以隐藏key
api_key = st.text_input(label='请输入你的key:', type='password')

# 希望把聊天数据进行保存, 如果只是使用一个普通变量来做存储的话, 那么当代码修改了, 数据会被重新初始化
# 可以使用状态数据
if 'messages' not in st.session_state:
    st.session_state['messages'] = [('ai', '你好, 我是你的个人小助理, 我叫小美')]

st.write('## 我的个人聊天机器人')
st.divider()

# 如果没有提供api_key的话, 那么其他区域就隐藏, 给用户一个提示
if not api_key:
    st.error('请提供你的大模型的api_key')
    st.stop()

# 遍历聊天列表
for role, content in st.session_state['messages']:
    st.chat_message(role).write(content)

# 输入问题的输入框
user_input = st.chat_input(placeholder='请输入')

if user_input:
    # 历史记录, 历史记录的数据结构二元组 (角色, 内容)
    _, history = st.session_state['messages'][-1]

    # 用户问题
    st.chat_message('human').write(user_input)
    # 把用户的提问添加进历史记录
    st.session_state['messages'].append(('human', user_input))

    # 输入问题后, 给AI一个思考的过程
    with st.spinner('思考中...'):
        # AI回答
        # history 把最后一个的记录, 每次问新问题时携带过去, 这样模型就可以记住之前的内容
        answer = get_answer(f'{history}, {user_input}')
        result = st.chat_message('ai').write_stream(answer)
        st.session_state['messages'].append(('ai', result))

```

二、网页摘要生成助手

1、当输入一个网址后，需要把页面的内容采集到，给到模型，模型把主要内容总结输出

2、bs4下载

网址: <https://www.runoob.com/python3/python-spider-beautifulsoup.html>

```
pip install beautifulsoup4
```

3、网页摘要提示词

```
user_prompt_template = '''
```

你是一个根据网页标题和内容生成摘要的优秀助手，请你根据我提供的标题和内容生成markdown格式的网页摘要。
注意：摘要中不要包含``markdown和``标签，将内容控制在1000个字符以内，尽可能用列表的形式呈现内容。

```
### 输出格式 ###
```

```
``markdown
```

```
## <标题>
```

```
### <第一部分>t
```

```
- <内容1>
```

```
- <内容2>
```

```
- <内容3>
```

```
### <第二部分>
```

```
- <内容1>
```

```
- <内容2>
```

```
- <内容3>
```

```
### ...
```

```
### 总结
```

```
<总结内容>
```

```
'''
```

```
### 网页标题和内容如下所示 ###
```

```
网页标题: {0}
```

```
主体内容: {1}
```

```
'''
```

4、网页数据采集函数封装

```
# 网页摘要数据采集函数封装
```

```
# url 表示的是需要采集数据的网址
```

```
def fetch_page(url):
```

```
    resp = requests.get(
```

```
        url=url,
```

```
        # 如果不携带请求头信息的话，直接请求网站，会做自动验证把你识别为程序（机器），这个有可能就直接给你返回一个空白页
```

```
        # use-agent 表示的是浏览器的版本信息，发送请求携带这个信息，要让网站以为咱们是通过浏览器正常访问，而不是程序（伪装）
```

```

headers={
    'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.3'
}
)
# 设置编码格式
resp.encoding = 'utf-8'
# 当请求成功返回结果
if resp.status_code == 200:
    # BeautifulSoup --- 解析网页
    # 参数1: 表示需要解析和处理的网站内容 resp.text
    # 参数2: 表示的是标记解析器
    # 支持的是css选择器
    # soup = BeautifulSoup(resp.text, 'html.parser')
    # print(soup.select_one('#head #s-top-left a'))
    # print('-' * 30)
    # print(soup.select('#head #s-top-left a'))
    # for elem in soup.select('#head #s-top-left a'):
    #     print(elem.get_text())

    soup = BeautifulSoup(resp.text, 'html.parser')
    for elem in soup.body(['script', 'link', 'nav', 'header', 'footer', 'form', 'input',
'button',
        'img', 'audio', 'video', 'area', 'canvas', 'map', 'object', 'embed']):
        # decompose() 彻底删除这里面包含的标记
        elem.decompose()

    return soup.title.text, soup.body.get_text(separator='\n', strip=True)

else:
    print(resp.status_code)

```

5、网页摘要生成助手

```

from fastapi.openapi.utils import generate_operation_summary
from openai import OpenAI
from dotenv import load_dotenv
from common import fetch_page, get_llm_response
import streamlit as st
load_dotenv()

user_prompt_template = '''
你是一个根据网页标题和内容生成摘要的优秀助手，请你根据我提供的标题和内容生成markdown格式的网页摘要。
注意：摘要中不要包含```markdown和```标签，将内容控制在1000个字符以内，尽可能用列表的形式呈现内容。

### 输出格式 ###
```markdown
<标题>

<第一部分>t
- <内容1>
- <内容2>
- <内容3>

```

```

<第二部分>
- <内容1>
- <内容2>
- <内容3>

...

总结
<总结内容>
'''

网页标题和内容如下所示
网页标题: {0}
主体内容: {1}
'''

def generate_summary(url):
 client = OpenAI(base_url='https://dashscope.aliyuncs.com/compatible-mode/v1')
 web_title, web_body = fetch_page(url)
 # user_prompt_template.format(web_title, web_body) 其实把提示词里面占位的字符给替换掉
 user_prompt = user_prompt_template.format(web_title, web_body)
 stream = get_llm_response(client, user_prompt=user_prompt, stream=True)
 for chunk in stream:
 yield chunk.choices[0].delta.content or ''

st.write('## 网页摘要生成助手')
st.divider()

result = ''

col1, _, col2 = st.columns([3, 1, 6])

with col1:
 url_input = st.text_input(label='要生成摘要网址', placeholder='请输入完整的网址')
 button = st.button('确定', type='primary')
 if button and url_input.strip():
 result = generate_summary(url_input)

with col2:
 if result:
 st.write('网页摘要:')
 st.write_stream(result)

```

## 三、langchain介绍

### 1、langchain

- 是一个做智能体的框架，提供了很多非要有用的工具（模块），可以提供开发效率
- langchain框架是2022年发布的，是一个开源的框架

## 2、如果不使用langchain它的话会存在什么问题？

- 使用OpenAI提示词每次都需要手动进行拼接
- 无法保存历史对话（使用列表进行存储，每次对话的时候需要把整个聊天历史列表给模型发送过去）
- 无法连接外部工具
- 无法连接向量数据库（可以把内容转换为向量，存储进向量数据库，每次执行的时候可以计算向量和向量之间距离，给出精准答案 --- RAG）
- 代码复用性非常差

## 3、使用langchain优势？

- 统一接口，轻松切换不同的模型
- 内置对话记忆，不要自己管理历史记录
- 内置支持工具，可以让AI查询天气、搜索网页...
- 内置向量数据库，可以快速实现RAG

## 4、官网文档地址

官网地址: <https://www.langchain.com/langchain>

官网文档: <https://python.langchain.com/docs/introduction/>

API文档: [https://python.langchain.com/api\\_reference/](https://python.langchain.com/api_reference/)

github地址: <https://github.com/langchain-ai/langchain>

## 5、目前最新稳定版本，1.2版本

## 6、注意点：大家学习langchain一定要注意语法的修改，要及时查看官网的动态

# 四、langchain组件

## 1、Schema（数据模型）

- Schema是langchain基础数据结构，定义了消息、文档格式

## 2、Prompts（提示模版）

- 专门用来生成提示词，然后发送给模型

## 3、Chains（链）

- 是一个可以将多个组件串联起来的工作流

## 4、Memory（记忆）

- 用于存储和管理对话历史，可以让AI记住前面的对话

## 5、Agent（代理）

- 可以把所有的组件组合起来，生成一个自主决策并生成内容的智能体

## 6、Models（模型）

- 是LLM的统一的标准接口（支持千问、OpenAI、Deepseek等等）

## 五、langchain环境安装

### 1、依赖安装

```
langchain依赖
pip install langchain

langchain社区依赖
pip install langchain-community

安装OpenAI集成包
pip install langchain-openai

安装deepseek集成包
pip install langchain-deepseek

安装通义千问集成包
pip install dashscope

一次性安装所有的依赖
pip install langchain langchain-community langchain-openai langchain-deepseek dashscope
```

### 2、配置 .env文件

```
OpenAI
OPENAI_API_KEY='sk-7f4016c1b34a43958d2a289d1c22f'

deepseek
DEEPSEEK_API_KEY='sk-57e80bf11bb54ac83d9973f1f607f'

qwen
DASHSCOPE_API_KEY='sk-7f4016c1b3958d2a289d1c30522f'
```

### 3、langchain环境安装验证

```
import langchain

print(langchain.__version__)
```

## 六、使用langchain调用模型

```
千问接口
from langchain_community.chat_models.tongyi import ChatTongyi

deepseek接口
from langchain_deepseek import ChatDeepSeek

标准接口（支持所有的 --- 目前千问不支持）
from langchain.chat_models import init_chat_model

OpenAI接口
from langchain.chat_models.openai import ChatOpenAI
```

```
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

resp = llm.invoke('你是谁? ').content
print(resp)
```